Symbolic Maneuver Planning for Orbital Rendezvous

This example derives closed-form maneuver planning equations for spacecraft proximity operations using the Clohessy-Wiltshire equations, then reuses those equations in downstream timing, safety, and sensitivity studies.

Workflow: Linear relative-motion ODEs -> `dsolve` analytic solution -> no-drift safety condition -> transfer delta-v equations -> `matlabFunction` code generation -> timing sweep, trajectory visualization, sensitivity analysis, and a compact uncertainty check

Requires: Symbolic Math Toolbox™

Copyright 2026 The MathWorks, Inc.

1. The Clohessy-Wiltshire Equations

The Clohessy-Wiltshire (CW) equations describe relative motion of a chaser spacecraft with respect to a target in a circular reference orbit. The coordinate frame is the Local-Vertical/Local-Horizontal (LVLH) frame centered on the target:

- x (R-bar): radial, outward from Earth center
- y (V-bar): along-track, in the direction of orbital velocity
- z (H-bar): cross-track, completing the right-hand frame

For a circular reference orbit with mean motion $n = \sqrt{\mu/a^3}$, the linearized equations of relative motion are:

$$\ddot{x} - 2n\dot{y} - 3n^2x = 0$$

$$\ddot{y} + 2n\dot{x} = 0$$

$$\ddot{z} + n^2z = 0$$

The in-plane (x-y) motion is coupled; cross-track (z) is independent.

```
syms n positive
syms t
% In-plane CW equations: x (radial), y (along-track)
syms x(t) y(t)
Dx = diff(x, t);
D2x = diff(x, t, 2);
```

```
Dy = diff(y, t);
D2y = diff(y, t, 2);
cw_x = D2x - 2*n*Dy - 3*n^2*x == 0;
cw_y = D2y + 2*n*Dx == 0;
% Cross-track CW equation: z (out-of-plane)
syms z(t)
Dz = diff(z, t);
D2z = diff(z, t, 2);
cw_z = D2z + n^2*z == 0;
```

Clohessy-Wiltshire equations:

```
cw_equations = [cw_x; cw_y; cw_z]
```

```
cw_equations(t) =
```

$$\begin{bmatrix} \frac{\partial^2}{\partial t^2} x(t) - 2\,n\,\frac{\partial}{\partial t} y(t) - 3\,x(t)\,n^2 = 0 \\ \frac{\partial^2}{\partial t^2} y(t) + 2\,n\,\frac{\partial}{\partial t} x(t) = 0 \\ \frac{\partial^2}{\partial t^2} z(t) + z(t)\,n^2 = 0 \end{bmatrix}$$

2. Analytic General Solution via `dsolve`

`dsolve` produces the exact closed-form solution of the coupled linear ODE system. The in-plane solution contains oscillatory terms plus the secular along-track drift term that matters for bounded proximity operations.

```
syms x0 y0 vx0 vy0 real
conds_xy = [x(0) == x0, y(0) == y0, Dx(0) == vx0, Dy(0) == vy0];
sol_xy = dsolve([cw_x, cw_y], conds_xy);
```

General in-plane solution $x(t)$:

```
x_sol = simplify(sol_xy.x)
```

```
x_sol =
```

$$\frac{2\ (\mathrm{vy}_0 + 2\,n\,x_0)}{n} - \frac{\cos(n\,t)\ (2\,\mathrm{vy}_0 + 3\,n\,x_0)}{n} + \frac{\mathrm{vx}_0 \sin(n\,t)}{n}$$

General in-plane solution $y(t)$:

```
y_sol = simplify(sol_xy.y)
```

y_sol =

$$\frac{2\,\mathrm{vx}_0 \cos(n\,t)}{n} - 3\,t\ (\mathrm{vy}_0 + 2\,n\,x_0) - \frac{2\,\mathrm{vx}_0 - n\,y_0}{n} + \frac{2\sin(n\,t)\ (2\,\mathrm{vy}_0 + 3\,n\,x_0)}{n}$$

```
syms z0 vz0 real
conds_z = [z(0) == z0, Dz(0) == vz0];
sol_z = dsolve(cw_z, conds_z);
```

Cross-track solution $z(t)$:

```
z_sol = simplify(sol_z)
```

z_sol =

$$z_0 \cos(n\,t) + \frac{\mathrm{vz}_0 \sin(n\,t)}{n}$$

No-Drift Condition

The secular drift is the coefficient of the term that grows linearly with time. Dividing by t and taking the limit removes the bounded sinusoidal terms and leaves the drift rate.

Along-track drift rate:

```
y_drift_rate = simplify(limit(y_sol/t, t, inf))
```

y_drift_rate = $-3\,\mathrm{vy}_0 - 6\,n\,x_0$

No-drift condition:

```
no_drift = solve(y_drift_rate == 0, vy0)
```

no_drift = $-2\,n\,x_0$

3. Impulsive Maneuver Delta-V

The transfer problem is: given initial relative position (x_0, y_0), desired final relative position (x_f, y_f), and transfer time T, what initial relative velocity reaches the endpoint?

```
syms T positive
syms xf yf real
x_at_T = subs(x_sol, t, T);
y_at_T = subs(y_sol, t, T);
endpoint_eqs = [x_at_T == xf, y_at_T == yf];
vel_sol = solve(endpoint_eqs, [vx0, vy0]);
```

Required departure velocity $v_{x0}(T, x_0, y_0, x_f, y_f)$:

```
vx_required = simplify(vel_sol.vx0)
```

vx_required =

$$_\frac{n\,(2\,y_0 - 2\,\mathrm{yf} - 2\,y_0\cos(T\,n) + 2\,\mathrm{yf}\,\cos(T\,n) - 4\,x_0\sin(T\,n) + 4\,\mathrm{xf}\,\sin(T\,n) - 3\,T\,n\,\mathrm{xf} + 3\,T\,n\,x_0\cos(T}{8\cos(T\,n) + 3\,T\,n\sin(T\,n) - 8}$$

Required departure velocity $v_{y0}(T, x_0, y_0, x_f, y_f)$:

```
vy_required = simplify(vel_sol.vy0)
```

vy_required =

$$_\frac{n\,(2\,\mathrm{xf} - 14\,x_0 + 14\,x_0\cos(T\,n) - 2\,\mathrm{xf}\,\cos(T\,n) - y_0\sin(T\,n) + \mathrm{yf}\,\sin(T\,n) + 6\,T\,n\,x_0\sin(T\,n))}{8\cos(T\,n) + 3\,T\,n\sin(T\,n) - 8}$$

Delta-V Magnitude

For a chaser initially station-keeping, the departure delta-v is the required relative velocity magnitude. The arrival delta-v is the velocity at $t = T$ that must be removed to stop at the target point.

```
dv_depart = simplify(sqrt(vx_required^2 + vy_required^2))
```

dv_depart =

$$\frac{n\sqrt{(2\,\mathrm{xf}-14\,x_0+14\,x_0\cos(T\,n)-2\,\mathrm{xf}\,\cos(T\,n)-y_0\sin(T\,n)+\mathrm{yf}\,\sin(T\,n)+6\,T\,n\,x_0\sin(T\,n))^2+(2\,y_0}}{|8\cos(T\,n)+3\,T\,n}$$

```
vx_sol = diff(x_sol, t);
vy_sol = diff(y_sol, t);
vx_arrival = simplify(subs(vx_sol, [t, vx0, vy0], [T, vx_required, vy_required]));
vy_arrival = simplify(subs(vy_sol, [t, vx0, vy0], [T, vx_required, vy_required]));
dv_arrive = simplify(sqrt(vx_arrival^2 + vy_arrival^2));
dv_total = simplify(dv_depart + dv_arrive)
```

dv_total =

$$n\left(\sqrt{(2\,\mathrm{xf}-14\,x_0+14\,x_0\cos(T\,n)-2\,\mathrm{xf}\,\cos(T\,n)-y_0\sin(T\,n)+\mathrm{yf}\,\sin(T\,n)+6\,T\,n\,x_0\sin(T\,n))^2+(2\,y_0}\right.$$

4. Reference Mission

Use an ISS-like servicing scenario: the chaser starts 1 km behind and 50 m below the target docking port in a 420 km circular reference orbit.

```
mission.altitudeKm = 420;
mission.earthRadiusKm = 6371;
mission.muKm = 398600.4418;
mission.semiMajorAxisKm = mission.earthRadiusKm + mission.altitudeKm;
mission.meanMotion = sqrt(mission.muKm / mission.semiMajorAxisKm^3);
mission.period = 2*pi / mission.meanMotion;
mission.initialPosition = [-50; -1000];
mission.finalPosition = [0; 0];
printReferenceMission(mission)
```

```
=== Reference Mission: ISS Proximity Operations ===
Orbit altitude:      420 km
Semi-major axis:     6791.0 km
Mean motion:         0.001128 rad/s
Orbital period:      92.8 min
Start position:      (-50, -1000) m  [R-bar, V-bar]
Target position:     (0, 0) m  [origin]
```

5. Code Generation for Reuse

Convert symbolic expressions into optimized MATLAB® functions. These generated functions are the handoff point from first-principles derivation to repeated engineering analysis.

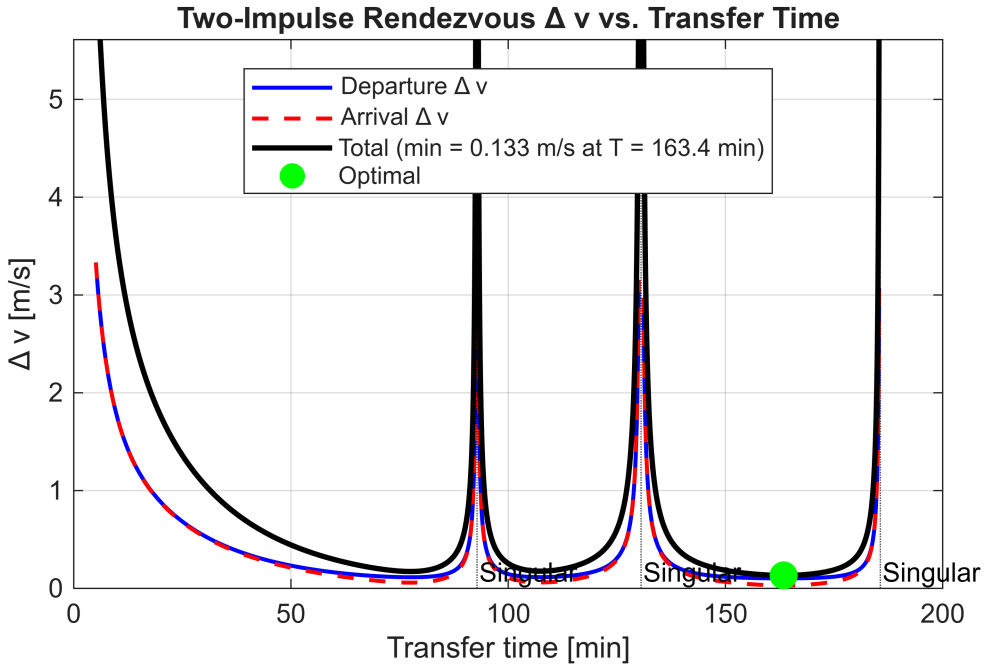
```
maneuverVars = {n, T, x0, y0, xf, yf};
matlabFunction(vx_required, vy_required, ...
    'File', 'departureVelocity', ...
    'Vars', maneuverVars, ...
    'Outputs', {'vx0_req', 'vy0_req'}, ...
    'Optimize', true);
matlabFunction(vx_arrival, vy_arrival, ...
    'File', 'arrivalVelocity', ...
    'Vars', maneuverVars, ...
    'Outputs', {'vx_arr', 'vy_arr'}, ...
    'Optimize', true);
matlabFunction(dv_depart, dv_arrive, ...
    'File', 'maneuverDeltaV', ...
    'Vars', maneuverVars, ...
    'Outputs', {'dv_dep', 'dv_arr'}, ...
    'Optimize', true);
trajectoryVars = {n, t, x0, y0, vx0, vy0};
matlabFunction(x_sol, y_sol, ...
    'File', 'cwTrajectory', ...
    'Vars', trajectoryVars, ...
    'Outputs', {'x_traj', 'y_traj'}, ...
    'Optimize', true);
fprintf(['Generated: departureVelocity.m\n' ...
    'Generated: arrivalVelocity.m\n' ...
    'Generated: maneuverDeltaV.m\n' ...
    'Generated: cwTrajectory.m\n'])
```

```
Generated: departureVelocity.m
Generated: arrivalVelocity.m
Generated: maneuverDeltaV.m
Generated: cwTrajectory.m
```

6. Fuel-Optimal Transfer Timing

The generated delta-v function evaluates the exact analytic expressions over thousands of transfer times in one vectorized call. Singular transfer times are marked and masked in the plot so the practical delta-v trade remains visible.

```
T_sweep = linspace(0.05*mission.period, 2*mission.period, 2000);
[dv_dep_sweep, dv_arr_sweep] = maneuverDeltaV(mission.meanMotion, T_sweep, ...
    mission.initialPosition(1), mission.initialPosition(2), ...
    mission.finalPosition(1), mission.finalPosition(2));
dv_total_sweep = dv_dep_sweep + dv_arr_sweep;
[optimal.dvTotal, optimal.index] = min(dv_total_sweep);
optimal.time = T_sweep(optimal.index);
optimal.dvDepart = dv_dep_sweep(optimal.index);
optimal.dvArrive = dv_arr_sweep(optimal.index);
plotDeltaVSweep(T_sweep, dv_dep_sweep, dv_arr_sweep, optimal, mission.meanMotion)
```



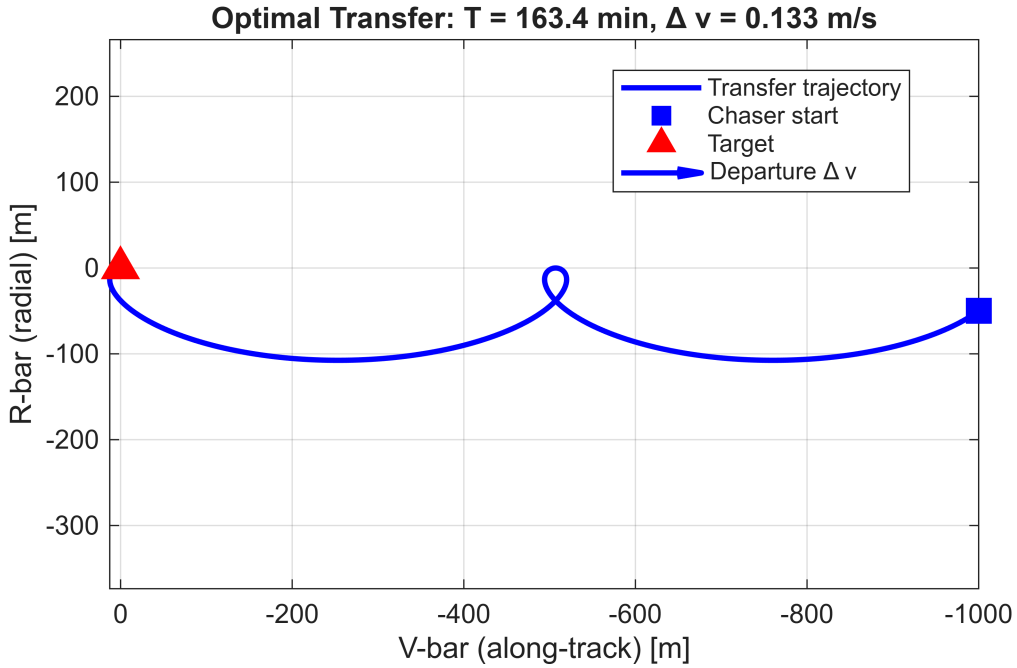

```
printFuelOptimalTransfer(optimal, mission)
```

```
=== Fuel-Optimal Transfer ===
Optimal transfer time: 163.4 min (1.76 orbits)
Departure delta-v:     0.1023 m/s
Arrival delta-v:       0.0303 m/s
Total delta-v:         0.1326 m/s
```

Optimal Transfer Trajectory

The same analytic trajectory solution becomes a reusable propagator after `matlabFunction`; here it evaluates the optimal transfer path directly.

```
[vx0_opt, vy0_opt] = departureVelocity(mission.meanMotion, optimal.time, ...
    mission.initialPosition(1), mission.initialPosition(2), ...
    mission.finalPosition(1), mission.finalPosition(2));
t_traj = linspace(0, optimal.time, 500);
[x_traj, y_traj] = cwTrajectory(mission.meanMotion, t_traj, ...
    mission.initialPosition(1), mission.initialPosition(2), vx0_opt, vy0_opt);
plotTransferTrajectory(x_traj, y_traj, vx0_opt, vy0_opt, optimal, mission)
```



7. Safety Ellipse Analysis

Substituting the no-drift condition into the symbolic solution gives the bounded relative-motion family. With zero radial velocity, the classical passive safety ellipse follows directly.

Bounded orbit $x(t)$ under $\dot{y}_0 = -2nx_0$:

```
x_bounded = simplify(subs(x_sol, vy0, no_drift))
```

```
x_bounded =
```

$$x_0 \cos(n\,t) + \frac{\mathrm{vx}_0 \sin(n\,t)}{n}$$

Bounded orbit $y(t)$ under $\dot{y}_0 = -2nx_0$:

```
y_bounded = simplify(subs(y_sol, vy0, no_drift))
```

```
y_bounded =
```

$$\frac{2\,\mathrm{vx}_0 \cos(n\,t)}{n} - \frac{2\,\mathrm{vx}_0 - n\,y_0}{n} - 2\,x_0 \sin(n\,t)$$

Pure hold case with $\dot{x}_0 = 0$:

```
x_pure_hold = simplify(subs(x_bounded, vx0, 0))
```

```
x_pure_hold = x_0 cos(n t)
```

```
y_pure_hold = simplify(subs(y_bounded, vx0, 0))
```

```
y_pure_hold = y_0 - 2 x_0 sin(n t)
```

The symbolic identity below is the ellipse equation $x^2/x_0^2 + (y - y_0)^2/(2x_0)^2 = 1$.

```
ellipse_identity = simplify(x_pure_hold^2/x0^2 + (y_pure_hold - y0)^2/(4*x0^2))
```

```
ellipse_identity = 1
```

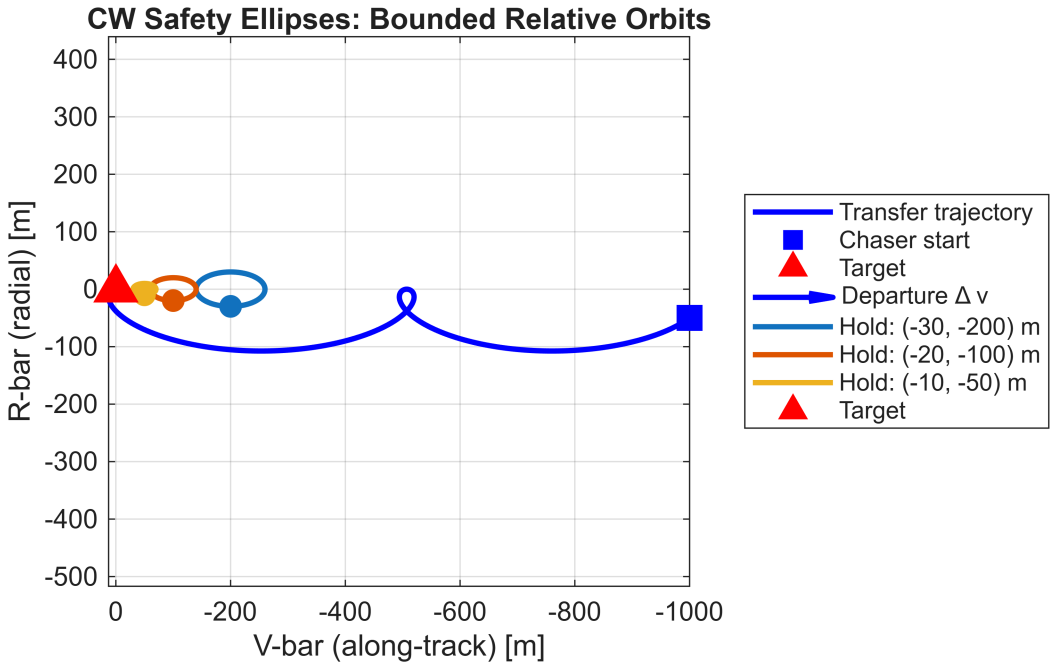
```
radial_semiaxis = abs(x0);
```

```
along_track_semiaxis = 2*abs(x0);
```

Visualize Safety Ellipses

The visualization is bookkeeping, so the plotting code is kept in a helper function while the symbolic ellipse derivation remains visible above.

```
plotSafetyEllipses(mission)
```



8. Sensitivity Analysis

Symbolic differentiation gives the exact Jacobian from departure velocity error to arrival position error. This is the local linear map that mission designers use to reason about burn execution sensitivity.

```
J_pos_vel = simplify(jacobian([x_at_T; y_at_T], [vx0, vy0]));
```

Maneuver sensitivity matrix $\partial(x_f, y_f)/\partial(v_{x0}, v_{y0})$:

```
J_pos_vel
```

```
J_pos_vel =
```

$$\begin{bmatrix} \frac{\sin(T\,n)}{n} & -\frac{2\,(\cos(T\,n)-1)}{n} \\ \frac{2\,(\cos(T\,n)-1)}{n} & \frac{4\sin(T\,n)}{n}-3\,T \end{bmatrix}$$

```
matlabFunction(J_pos_vel, ...
    'File', 'maneuverSensitivity', ...
    'Vars', {n, T}, ...
    'Outputs', {'J'}, ...
    'Optimize', true);
fprintf('Generated: maneuverSensitivity.m\n')
```

```
Generated: maneuverSensitivity.m
```

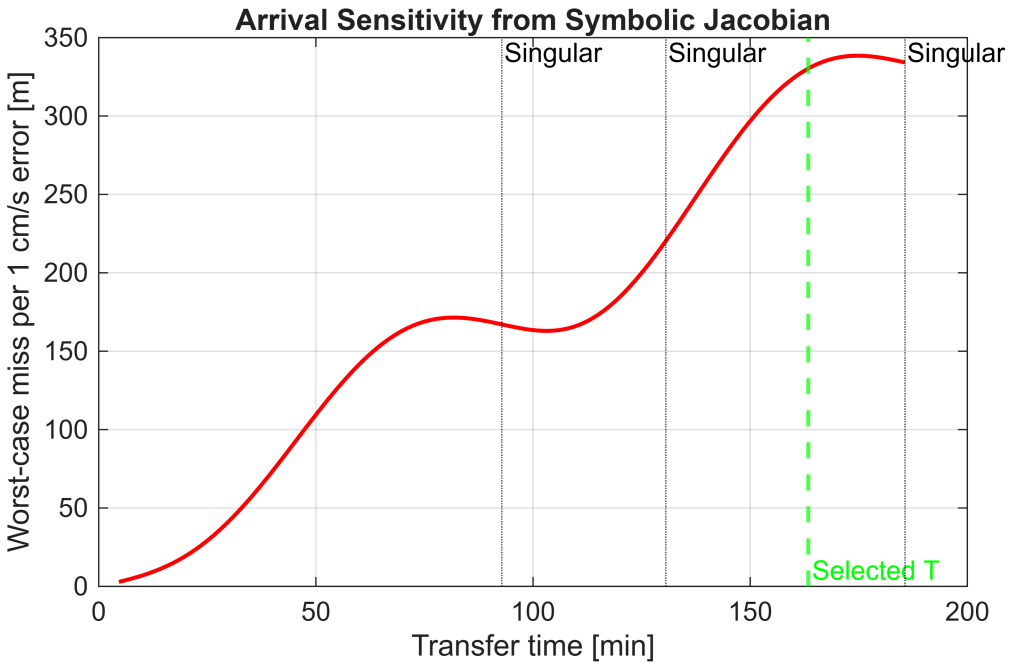
Sensitivity Across Transfer Time

Reusing the generated sensitivity function over the same timing grid shows how departure burn errors are amplified by transfer timing. The selected fuel-optimal transfer time is marked for context.

```
J_opt = maneuverSensitivity(mission.meanMotion, optimal.time);
printSensitivitySummary(J_opt, optimal)
```

```
=== Maneuver Sensitivity at T = 163.4 min ===
Sensitivity matrix [m per m/s]:
  [ -884.64   1661.16]
  [-1661.16  -32945.73]
Singular values: 33027.10, 966.02 m/(m/s)
Worst-case: 1 cm/s error -> 330.27 m miss distance
Best-case:  1 cm/s error -> 9.66 m miss distance
```

```
J_sweep = arrayfun(@(time) maneuverSensitivity(mission.meanMotion, time), T_sweep, ...
    'UniformOutput', false);
worst_amp = cellfun(@(J) max(svd(J)), J_sweep);
plotSensitivitySweep(T_sweep, worst_amp, optimal, mission.meanMotion)
```



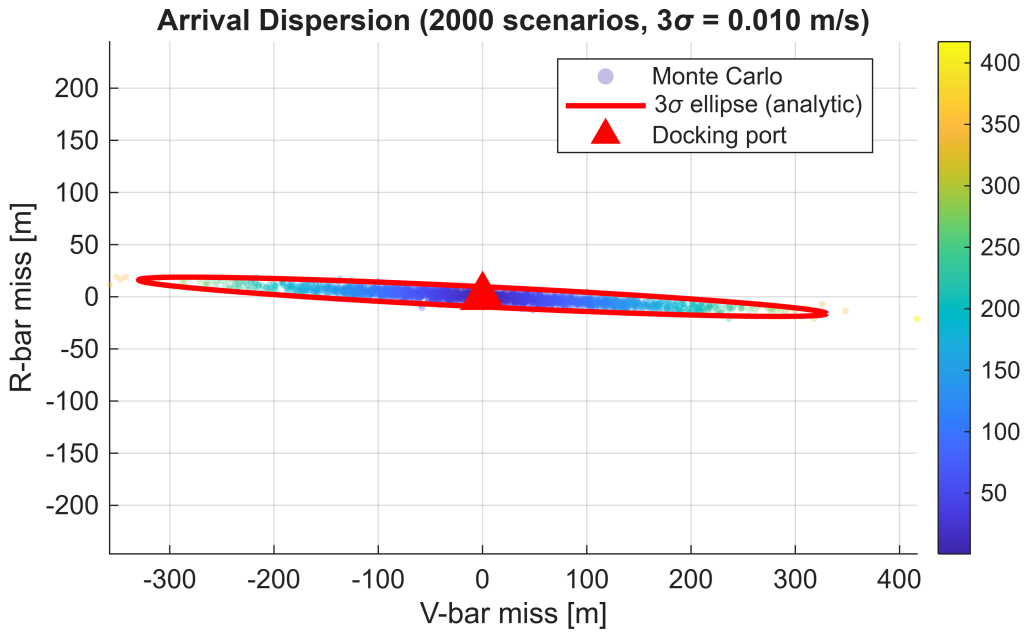
9. Compact Uncertainty Reuse Check

A small Monte Carlo check shows another downstream use of the generated equations. It is intentionally framed as a reuse demonstration, not as a claim that this simple two-impulse transfer meets final docking dispersions.

```
scenarioCount = 2000;
sigmaVelocity = 0.01/3;  % 1 cm/s at 3-sigma, per velocity axis
dispersion = runVelocityErrorMonteCarlo(mission, optimal, sigmaVelocity, scenarioCount);
printDispersionSummary(dispersion)
```

```
=== Compact Velocity-Error Reuse Check ===
Transfer time:                    163.4 min
Scenarios:                        2000
Velocity error, 3-sigma per axis: 0.0100 m/s
Nominal departure velocity:       (-0.0605, 0.0825) m/s
Mean miss distance:               88.242 m
99th percentile miss distance:    286.471 m
Analytic 3-sigma semi-axes:       330.271 m, 9.660 m
```

```
plotDispersionMonteCarlo(dispersion)
```



Helper Functions

The helper functions below keep plotting, reporting, and random sampling out of the symbolic derivation path.

```
function printReferenceMission(mission)
arguments
    mission (1,1) struct
end
fprintf(['=== Reference Mission: ISS Proximity Operations ===\n' ...
    'Orbit altitude:      %d km\n' ...
    'Semi-major axis:     %.1f km\n' ...
    'Mean motion:         %.6f rad/s\n' ...
    'Orbital period:      %.1f min\n' ...
    'Start position:      (%.0f, %.0f) m  [R-bar, V-bar]\n' ...
    'Target position:     (%.0f, %.0f) m  [origin]\n'], ...
```

```
    mission.altitudeKm, ...
    mission.semiMajorAxisKm, ...
    mission.meanMotion, ...
    mission.period/60, ...
    mission.initialPosition(1), mission.initialPosition(2), ...
    mission.finalPosition(1), mission.finalPosition(2))
end
function plotDeltaVSweep(T_sweep, dv_dep_sweep, dv_arr_sweep, optimal, meanMotion)
arguments
    T_sweep (1,:) double
    dv_dep_sweep (1,:) double
    dv_arr_sweep (1,:) double
    optimal (1,1) struct
    meanMotion (1,1) double {mustBePositive}
end
dvTotalSweep = dv_dep_sweep + dv_arr_sweep;
[plotMask, singularTimes] = transferPlotMask(T_sweep, dvTotalSweep, optimal, meanMotion);
dvDepPlot = dv_dep_sweep;
dvArrPlot = dv_arr_sweep;
dvTotalPlot = dvTotalSweep;
dvDepPlot(~plotMask) = NaN;
dvArrPlot(~plotMask) = NaN;
dvTotalPlot(~plotMask) = NaN;
plot(T_sweep/60, dvDepPlot, 'b-', 'LineWidth', 1.5)
hold on
plot(T_sweep/60, dvArrPlot, 'r--', 'LineWidth', 1.5)
plot(T_sweep/60, dvTotalPlot, 'k-', 'LineWidth', 2)
plot(optimal.time/60, optimal.dvTotal, 'go', 'MarkerSize', 10, 'MarkerFaceColor', 'g')
for idx = 1:numel(singularTimes)
    xline(singularTimes(idx)/60, 'k:', 'Singular', ...
        'HandleVisibility', 'off', ...
        'LabelOrientation', 'horizontal', ...
        'LabelVerticalAlignment', 'bottom')
end
hold off
xlabel('Transfer time [min]')
ylabel('\Delta v [m/s]')
```

```
title('Two-Impulse Rendezvous \Delta v vs. Transfer Time')
legend('Departure \Delta v', 'Arrival \Delta v', ...
    sprintf('Total (min = %.3f m/s at T = %.1f min)', optimal.dvTotal, optimal.time/60), ...
    'Optimal', 'Location', 'best')
ylim([0, practicalDeltaVLimit(dvTotalSweep, plotMask, optimal)])
grid on
end
function [plotMask, singularTimes] = transferPlotMask(T_sweep, dvTotalSweep, optimal, meanMotion)
arguments
    T_sweep (1,:) double
    dvTotalSweep (1,:) double
    optimal (1,1) struct
    meanMotion (1,1) double {mustBePositive}
end
theta = meanMotion*T_sweep;
transferDenominator = 8*cos(theta) + 3*theta.*sin(theta) - 8;
denominatorMask = abs(transferDenominator) > 0.1;
finiteMask = isfinite(dvTotalSweep);
initialMask = finiteMask & denominatorMask;
plotLimit = practicalDeltaVLimit(dvTotalSweep, initialMask, optimal);
plotMask = initialMask & (dvTotalSweep <= plotLimit);
singularTimes = findTransferSingularities(T_sweep, meanMotion);
end
function upperLimit = practicalDeltaVLimit(dvTotalSweep, plotMask, optimal)
arguments
    dvTotalSweep (1,:) double
    plotMask (1,:) logical
    optimal (1,1) struct
end
validDeltaV = dvTotalSweep(plotMask & isfinite(dvTotalSweep) & dvTotalSweep > 0);
if isempty(validDeltaV)
    upperLimit = 10*optimal.dvTotal;
else
    upperLimit = max(1.25*prctile(validDeltaV, 98), 5*optimal.dvTotal);
end
end
function singularTimes = findTransferSingularities(T_sweep, meanMotion)
```

```
arguments
    T_sweep (1,:) double
    meanMotion (1,1) double {mustBePositive}
end
thetaRange = meanMotion*[min(T_sweep), max(T_sweep)];
thetaGrid = linspace(thetaRange(1), thetaRange(2), 5000);
denominator = @(theta) 8*cos(theta) + 3*theta.*sin(theta) - 8;
denominatorValues = denominator(thetaGrid);
crossingIdx = find(denominatorValues(1:end-1).*denominatorValues(2:end) <= 0);
singularTheta = zeros(1, 0);
for idx = crossingIdx
    thetaLeft = thetaGrid(idx);
    thetaRight = thetaGrid(idx + 1);
    valueLeft = denominatorValues(idx);
    valueRight = denominatorValues(idx + 1);
    if valueLeft == 0
        thetaRoot = thetaLeft;
    elseif valueRight == 0
        thetaRoot = thetaRight;
    else
        thetaRoot = fzero(denominator, [thetaLeft, thetaRight]);
    end
    if isempty(singularTheta) || all(abs(thetaRoot - singularTheta) > 1e-5)
        singularTheta(end + 1) = thetaRoot; %#ok<AGROW>
    end
end
endpointTheta = thetaRange(abs(denominator(thetaRange)) < 1e-8);
for thetaRoot = endpointTheta
    if isempty(singularTheta) || all(abs(thetaRoot - singularTheta) > 1e-5)
        singularTheta(end + 1) = thetaRoot; %#ok<AGROW>
    end
end
singularTimes = sort(singularTheta/meanMotion);
end
function printFuelOptimalTransfer(optimal, mission)
arguments
    optimal (1,1) struct
```

```
    mission (1,1) struct
end
fprintf(['\n=== Fuel-Optimal Transfer ===\n' ...
    'Optimal transfer time: %.1f min (%.2f orbits)\n' ...
    'Departure delta-v:     %.4f m/s\n' ...
    'Arrival delta-v:       %.4f m/s\n' ...
    'Total delta-v:         %.4f m/s\n'], ...
    optimal.time/60, optimal.time/mission.period, ...
    optimal.dvDepart, optimal.dvArrive, optimal.dvTotal)
end
function plotTransferTrajectory(x_traj, y_traj, vx0_opt, vy0_opt, optimal, mission)
arguments
    x_traj (1,:) double
    y_traj (1,:) double
    vx0_opt (1,1) double
    vy0_opt (1,1) double
    optimal (1,1) struct
    mission (1,1) struct
end
plot(y_traj, x_traj, 'b-', 'LineWidth', 2)
hold on
plot(mission.initialPosition(2), mission.initialPosition(1), 'bs', ...
    'MarkerSize', 12, 'MarkerFaceColor', 'b')
plot(mission.finalPosition(2), mission.finalPosition(1), 'r^', ...
    'MarkerSize', 12, 'MarkerFaceColor', 'r')
quiver(mission.initialPosition(2), mission.initialPosition(1), vy0_opt*100, vx0_opt*100, 0, ...
    'b', 'LineWidth', 2, 'MaxHeadSize', 0.5)
hold off
xlabel('V-bar (along-track) [m]')
ylabel('R-bar (radial) [m]')
title(sprintf('Optimal Transfer: T = %.1f min, \\Delta v = %.3f m/s', ...
    optimal.time/60, optimal.dvTotal))
legend('Transfer trajectory', 'Chaser start', 'Target', 'Departure \Delta v', ...
    'Location', 'best')
set(gca, 'XDir', 'reverse')
grid on
axis equal
```

```
end
function plotSafetyEllipses(mission)
arguments
    mission (1,1) struct
end
holdDistances = [-200, -100, -50];
radialOffsets = [-30, -20, -10];
tEllipse = linspace(0, mission.period, 500);
colors = lines(numel(holdDistances));
hold on
for idx = 1:numel(holdDistances)
    x0Hold = radialOffsets(idx);
    y0Hold = holdDistances(idx);
    vy0Hold = -2*mission.meanMotion*x0Hold;
    [xEllipse, yEllipse] = cwTrajectory(mission.meanMotion, tEllipse, ...
        x0Hold, y0Hold, 0, vy0Hold);
    plot(yEllipse, xEllipse, '-', 'Color', colors(idx,:), 'LineWidth', 2, ...
        'DisplayName', sprintf('Hold: (%.0f, %.0f) m', x0Hold, y0Hold))
    plot(y0Hold, x0Hold, 'o', 'Color', colors(idx,:), ...
        'MarkerSize', 8, 'MarkerFaceColor', colors(idx,:), ...
        'HandleVisibility', 'off')
end
plot(0, 0, 'r^', 'MarkerSize', 14, 'MarkerFaceColor', 'r', ...
    'DisplayName', 'Target')
hold off
xlabel('V-bar (along-track) [m]')
ylabel('R-bar (radial) [m]')
title('CW Safety Ellipses: Bounded Relative Orbits')
legend('Location', 'eastoutside')
set(gca, 'XDir', 'reverse')
grid on
axis equal
end
function printSensitivitySummary(J_opt, optimal)
arguments
    J_opt (2,2) double
    optimal (1,1) struct
```

```
end
singularValues = svd(J_opt);
fprintf(['\n=== Maneuver Sensitivity at T = %.1f min ===\n' ...
    'Sensitivity matrix [m per m/s]:\n' ...
    '  [%8.2f  %8.2f]\n' ...
    '  [%8.2f  %8.2f]\n' ...
    'Singular values: %.2f, %.2f m/(m/s)\n' ...
    'Worst-case: 1 cm/s error -> %.2f m miss distance\n' ...
    'Best-case:  1 cm/s error -> %.2f m miss distance\n'], ...
    optimal.time/60, ...
    J_opt(1,1), J_opt(1,2), ...
    J_opt(2,1), J_opt(2,2), ...
    singularValues(1), singularValues(2), ...
    singularValues(1)*0.01, singularValues(2)*0.01)
end
function plotSensitivitySweep(T_sweep, worst_amp, optimal, meanMotion)
arguments
    T_sweep (1,:) double
    worst_amp (1,:) double
    optimal (1,1) struct
    meanMotion (1,1) double {mustBePositive}
end
singularTimes = findTransferSingularities(T_sweep, meanMotion);
plot(T_sweep/60, worst_amp*0.01, 'r-', 'LineWidth', 1.5)
hold on
xline(optimal.time/60, 'g--', 'Selected T', ...
    'LineWidth', 1.5, ...
    'LabelOrientation', 'horizontal', ...
    'LabelVerticalAlignment', 'bottom')
for idx = 1:numel(singularTimes)
    xline(singularTimes(idx)/60, 'k:', 'Singular', ...
        'HandleVisibility', 'off', ...
        'LabelOrientation', 'horizontal', ...
        'LabelVerticalAlignment', 'top')
end
hold off
xlabel('Transfer time [min]')
```

```
ylabel('Worst-case miss per 1 cm/s error [m]')
title('Arrival Sensitivity from Symbolic Jacobian')
grid on
end
function dispersion = runVelocityErrorMonteCarlo(mission, optimal, sigmaVelocity, scenarioCount)
arguments
    mission (1,1) struct
    optimal (1,1) struct
    sigmaVelocity (1,1) double {mustBePositive}
    scenarioCount (1,1) double {mustBeInteger, mustBePositive}
end
rng(42)
[vxNominal, vyNominal] = departureVelocity(mission.meanMotion, optimal.time, ...
    mission.initialPosition(1), mission.initialPosition(2), ...
    mission.finalPosition(1), mission.finalPosition(2));
velocityErrors = sigmaVelocity*randn(scenarioCount, 2);
vxSamples = vxNominal + velocityErrors(:,1);
vySamples = vyNominal + velocityErrors(:,2);
[xFinal, yFinal] = cwTrajectory(mission.meanMotion, optimal.time, ...
    mission.initialPosition(1), mission.initialPosition(2), vxSamples, vySamples);
missRadial = xFinal - mission.finalPosition(1);
missAlong = yFinal - mission.finalPosition(2);
missDistance = sqrt(missRadial.^2 + missAlong.^2);
J = maneuverSensitivity(mission.meanMotion, optimal.time);
positionCovariance = J*(sigmaVelocity^2*eye(2))*J';
[eigVectors, eigValues] = eig(positionCovariance);
[semiAxes, order] = sort(3*sqrt(diag(eigValues)), 'descend');
eigVectors = eigVectors(:, order);
theta = linspace(0, 2*pi, 200);
ellipsePoints = eigVectors*diag(semiAxes)*[cos(theta); sin(theta)];
dispersion.transferTime = optimal.time;
dispersion.sigmaVelocity = sigmaVelocity;
dispersion.scenarioCount = scenarioCount;
dispersion.nominalVelocity = [vxNominal; vyNominal];
dispersion.missRadial = missRadial;
dispersion.missAlong = missAlong;
dispersion.missDistance = missDistance;
```

```
dispersion.semiAxes = semiAxes;
dispersion.ellipsePoints = ellipsePoints;
end
function printDispersionSummary(dispersion)
arguments
    dispersion (1,1) struct
end
fprintf(['\n=== Compact Velocity-Error Reuse Check ===\n' ...
    'Transfer time:                    %.1f min\n' ...
    'Scenarios:                        %d\n' ...
    'Velocity error, 3-sigma per axis: %.4f m/s\n' ...
    'Nominal departure velocity:       (%.4f, %.4f) m/s\n' ...
    'Mean miss distance:               %.3f m\n' ...
    '99th percentile miss distance:    %.3f m\n' ...
    'Analytic 3-sigma semi-axes:       %.3f m, %.3f m\n'], ...
    dispersion.transferTime/60, ...
    dispersion.scenarioCount, ...
    3*dispersion.sigmaVelocity, ...
    dispersion.nominalVelocity(1), dispersion.nominalVelocity(2), ...
    mean(dispersion.missDistance), ...
    quantile(dispersion.missDistance, 0.99), ...
    dispersion.semiAxes(1), dispersion.semiAxes(2))
end
function plotDispersionMonteCarlo(dispersion)
arguments
    dispersion (1,1) struct
end
tiledlayout(1, 1)
nexttile
scatter(dispersion.missAlong, dispersion.missRadial, 6, dispersion.missDistance, ...
    'filled', 'MarkerFaceAlpha', 0.3)
hold on
plot(dispersion.ellipsePoints(2,:), dispersion.ellipsePoints(1,:), 'r-', 'LineWidth', 2)
plot(0, 0, 'r^', 'MarkerSize', 14, 'MarkerFaceColor', 'r')
hold off
colorbar
colormap(parula)
```

```
xlabel('V-bar miss [m]')
ylabel('R-bar miss [m]')
title(sprintf('Arrival Dispersion (%d scenarios, 3\\sigma = %.3f m/s)', ...
    dispersion.scenarioCount, 3*dispersion.sigmaVelocity))
legend('Monte Carlo', '3\sigma ellipse (analytic)', 'Docking port', ...
    'Location', 'best')
grid on
axis equal
end
```